

ADR-005-authentication-system

ADR-005: Authentication System

Status

Accepted

Date

2026-01-08

Context

HelixCode is a multi-client platform supporting multiple interfaces: - REST API (web applications, integrations) - CLI (command-line development) - TUI (terminal user interface) - Desktop application - Mobile applications (iOS, Android) - WebSocket (real-time communications)

The authentication system must:

1. **Support multiple client types:** Each client has different security characteristics
2. **Provide session management:** Track active sessions across devices
3. **Enable secure password storage:** Protect credentials at rest
4. **Support token-based authentication:** For API and stateless clients
5. **Enable MFA:** Multi-factor authentication for enterprise security
6. **Audit authentication events:** Track login attempts and security events
7. **Handle session lifecycle:** Creation, validation, expiration, revocation
8. **Scale horizontally:** Support distributed deployments

Decision

We implemented a hybrid authentication system combining JWT tokens for stateless API access with session-based authentication for stateful clients.

Core Authentication Service

```
type AuthService struct {  
    config AuthConfig  
    db      AuthRepository  
}  
  
type AuthConfig struct {  
    JWTSecret      string  
    TokenExpiry    time.Duration  
    SessionExpiry  time.Duration  
    BcryptCost     int
```

```

    Argon2Time      uint32
    Argon2Memory    uint32
    Argon2Threads   uint8
    Argon2KeyLength uint32
}

```

User Model

```

type User struct {
    ID          uuid.UUID `json:"id"`
    Username    string   `json:"username"`
    Email       string   `json:"email"`
    DisplayName string   `json:"display_name"`
    IsActive    bool     `json:"is_active"`
    IsVerified  bool     `json:"is_verified"`
    MFAEnabled  bool     `json:"mfa_enabled"`
    LastLogin   time.Time `json:"last_login"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

```

Session Model

```

type Session struct {
    ID          uuid.UUID `json:"id"`
    UserID      uuid.UUID `json:"user_id"`
    SessionToken string   `json:"session_token"`
    ClientType  string   `json:"client_type"` // terminal_ui,
                    cli, rest_api, mobile_ios, mobile_android
    IPAddress   net.IP    `json:"ip_address"`
    UserAgent   string   `json:"user_agent"`
    ExpiresAt   time.Time `json:"expires_at"`
    CreatedAt   time.Time `json:"created_at"`
}

```

Password Security

Dual Algorithm Support: - **Primary:** bcrypt for new passwords (configurable cost factor) -
Fallback: Argon2id for migrated passwords

```

func (s *AuthService) hashPassword(password string) (string,
    error) {
    hash, err := bcrypt.GenerateFromPassword([]byte(password),
        s.config.BcryptCost)
    return string(hash), err
}

func (s *AuthService) verifyPassword(password, hash string) bool
{

```

```

// Try bcrypt first
err := bcrypt.CompareHashAndPassword([]byte(hash),
    []byte(password))
if err == nil {
    return true
}
// Fallback to Argon2
return s.verifyArgon2Password(password, hash)
}

```

Argon2 Parameters: - Time cost: 1 iteration - Memory: 64 MB - Parallelism: 4 threads - Key length: 32 bytes

JWT Token System

JWT tokens for API authentication:

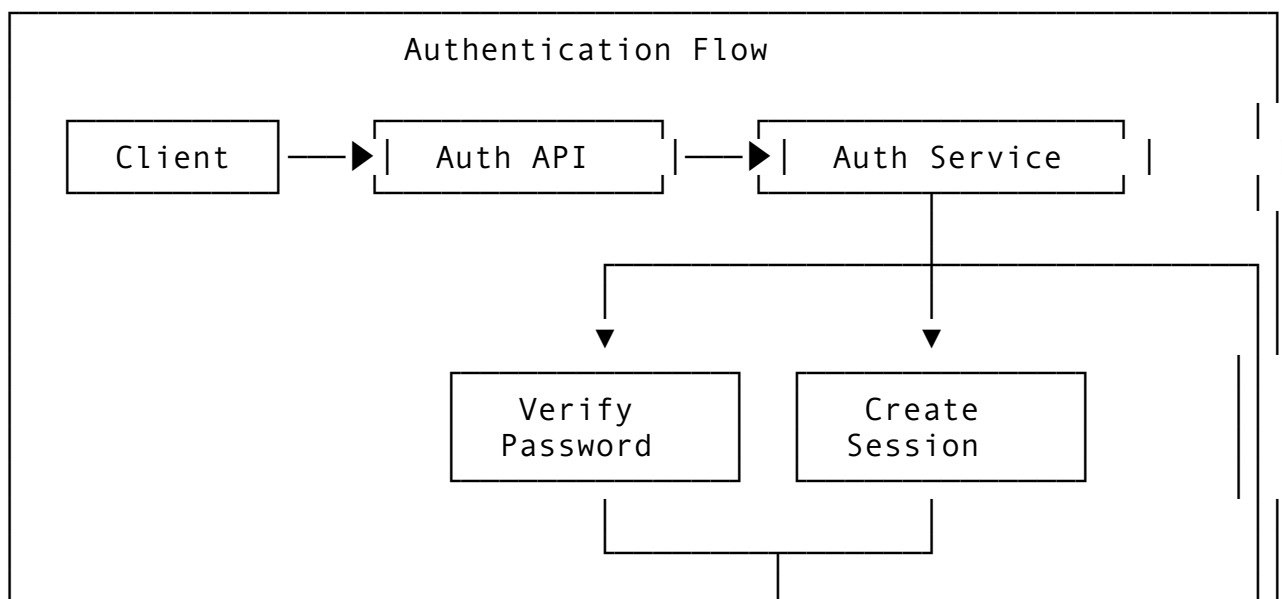
```

func (s *AuthService) GenerateJWT(user *User) (string, error) {
    claims := jwt.MapClaims{
        "user_id": user.ID.String(),
        "username": user.Username,
        "email":    user.Email,
        "exp":      time.Now().Add(s.config.TokenExpiry).Unix(),
        "iat":      time.Now().Unix(),
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(s.config.JWTSecret))
}

```

Two verification methods: 1. VerifyJWT: Fast, returns minimal user from claims 2. VerifyJWTWithDB: Slower, returns complete user with database lookup

Authentication Flow



▼

Return
JWT/Session

Session Management

```
// Create session on login
session := &Session{
    ID:          uuid.New(),
    UserID:      user.ID,
    SessionToken: s.generateSessionToken(),
    ClientType:  clientType,
    IPAddress:   ip,
    UserAgent:   userAgent,
    ExpiresAt:   time.Now().Add(s.config.SessionExpiry),
}

// Session token generation (32 bytes, URL-safe base64)
func (s *AuthService) generateSessionToken() (string, error) {
    bytes := make([]byte, 32)
    _, err := rand.Read(bytes)
    return base64.URLEncoding.EncodeToString(bytes), err
}
```

Security Features

1. **Constant-time comparison:** Prevents timing attacks on password verification
2. **Session IP tracking:** Detect session hijacking
3. **User agent tracking:** Identify device changes
4. **Account deactivation support:** Disabled accounts cannot authenticate
5. **Session expiration:** Automatic cleanup of expired sessions

Repository Interface

```
type AuthRepository interface {
    CreateUser(ctx context.Context, user *User, passwordHash
        string) error
    GetUserByUsername(ctx context.Context, username string)
        (*User, string, error)
    GetUserByEmail(ctx context.Context, email string) (*User,
        string, error)
    GetUserByID(ctx context.Context, id uuid.UUID) (*User, error)
    UpdateUserLastLogin(ctx context.Context, id uuid.UUID) error
    UpdateUser(ctx context.Context, userID uuid.UUID,
        displayName, email string) (*User, error)
    DeleteUser(ctx context.Context, userID uuid.UUID) error
    CreateSession(ctx context.Context, session *Session) error
}
```

```

    GetSession(ctx context.Context, token string) (*Session,
        error)
    DeleteSession(ctx context.Context, token string) error
    DeleteUserSessions(ctx context.Context, userID uuid.UUID)
        error
}

```

Consequences

Positive

1. **Flexibility:** Supports both token and session-based auth
2. **Security:** Strong password hashing with bcrypt/Argon2
3. **Multi-client:** Different client types handled appropriately
4. **Auditability:** Sessions track client type, IP, user agent
5. **Scalability:** JWT tokens are stateless for horizontal scaling
6. **Backwards Compatibility:** Dual algorithm support for migrations
7. **Session Control:** Users can revoke individual or all sessions

Negative

1. **Token Revocation:** JWT cannot be revoked before expiry without blacklist
2. **Secret Management:** JWT secret must be carefully managed
3. **Session Storage:** Session-based auth requires database access
4. **Complexity:** Supporting multiple auth methods increases complexity

Neutral

1. **MFA:** Currently flagged but not fully implemented
2. **OAuth:** Future consideration for third-party auth

Alternatives Considered

Alternative 1: OAuth 2.0 Only

Description: Use OAuth 2.0 with authorization code flow for all clients.

Pros: - Industry standard - Third-party auth support - Token refresh flow - Well-documented

Cons: - Complex for CLI clients - Requires redirect flow - Overkill for internal auth - Additional dependencies

Why Rejected: CLI and TUI clients need simpler authentication. OAuth can be added later for third-party integrations.

Alternative 2: Session-Only Authentication

Description: Use server-side sessions exclusively, no JWT.

Pros: - Immediate session revocation - No token expiry issues - Simpler implementation - Centralized session control

Cons: - Database lookup for every request - Harder to scale horizontally - Session affinity requirements - Higher latency

Why Rejected: API clients benefit from stateless JWT tokens. Hybrid approach provides flexibility.

Alternative 3: Passwordless (Magic Links/WebAuthn)

Description: Eliminate passwords in favor of magic links or WebAuthn.

Pros: - No password management - Phishing resistant (WebAuthn) - Better user experience - Modern approach

Cons: - Email dependency (magic links) - Hardware dependency (WebAuthn) - Not all clients support - Complex implementation

Why Rejected: Many HelixCode use cases require offline or automated access where passwordless doesn't work well. Can be added as an option later.

Alternative 4: API Keys Only

Description: Use long-lived API keys for all authentication.

Pros: - Simple to implement - No expiry management - Easy for automation - Stateless

Cons: - No session management - Hard to revoke - Security risk if leaked - No user context changes

Why Rejected: Interactive clients need session tracking for security. API keys can be a supplementary option.

Implementation Notes

- Auth implementation in `internal/auth/auth.go`
- Database integration in `internal/auth/auth_db.go`
- Default config provides secure defaults
- Environment variable `HELIX_AUTH_JWT_SECRET` for production secret
- Session tokens are 32 bytes of cryptographic randomness

Security Considerations

1. **Password Requirements:** Minimum 8 characters enforced
2. **Email Validation:** Basic format validation
3. **Username Requirements:** 3-255 characters
4. **Session Token Entropy:** 256 bits (32 bytes)
5. **JWT Signing:** HMAC-SHA256

Related Decisions

- ADR-006: Database Schema Design (user and session storage)
- ADR-008: Mobile Platform Strategy (mobile authentication flow)

References

- /run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/auth/auth.go
- /run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/auth/auth_db.go